VISUALIZATION

Visualization is a way of representing. Data visualization is representing of data in a graphical or pictorial format for better understanding of data. Visualization gives a good idea of data and the trends in it.

In python, data visualization has multiple libraries like matplotlib, seaborn, plotly etc.

Let us dive deeper into matplotlib and seaborn.

MATPLOTLIB

Matplotlib, introduced in 2002 by John Hunter. It is for plotting 2D plots which is built on numpy. A plot of matplotlib contains:

1 Figure

2 Axes

3 Axis

4 Artists

Figure

Figure is the container in which plots are present. It can have a single plot or multiple plots.

Axes

Axes are the plots, and they are artist attached to the figure. A plot can have 2 or 3 axes. set_xlabel(), set_ylabel() are the functions used to set the labels of x and y respectively.

Axis

It is responsible for generatingticks or the limits on the axes.

Artists

The whatever content visible in a figure are the artists.

PYPLOT

Pyplot is a sub library of matplotlib where all the utilities lie under. It has different types of plots including bar graphs, scatter plots, pie charts, histograms, area charts.

We import pyplot from matplotlib as following:

```
import matplotlib.pyplot as plt
```

We also import numpy as a support for the matplotlib :

```
import numpy as np
```

Some of the plots in matplotlib:

LINE CHART

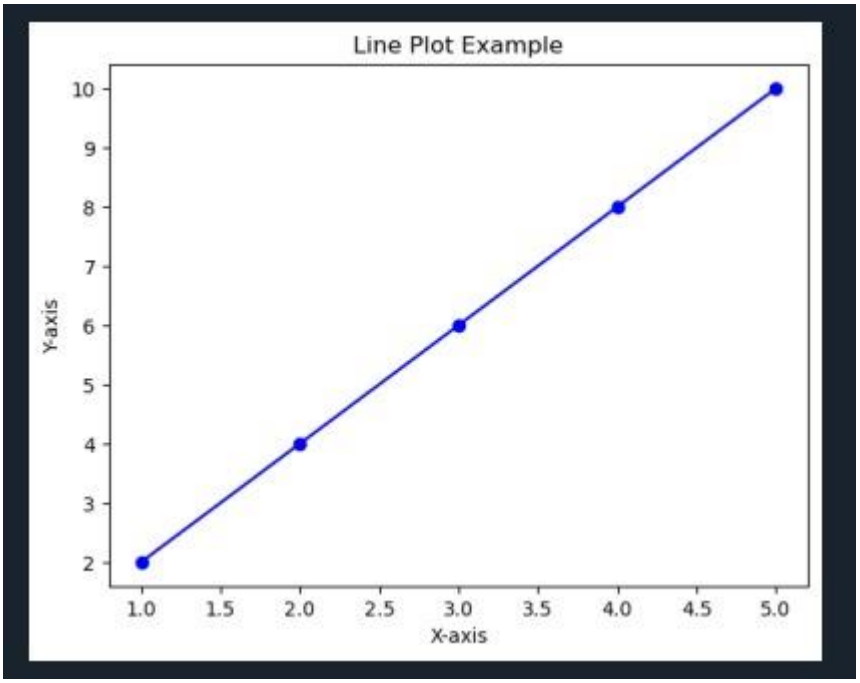
Line plots are the basic charts where dot consecutive points are connected by a continuous line.

The default plot() is used for line charts.

Code snippet:

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]
plt.plot(x, y, color='blue', marker='o')
plt.title("Line Plot Example")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.show()
```

Output:



Description:

np.array() will generate an array in the specified range.

plot() for the plotting the line chart.

title() for defining the title, ylabel() and xlabel () for labelling y and x axis respectively.

show() displays the graph.

BARGRAPH

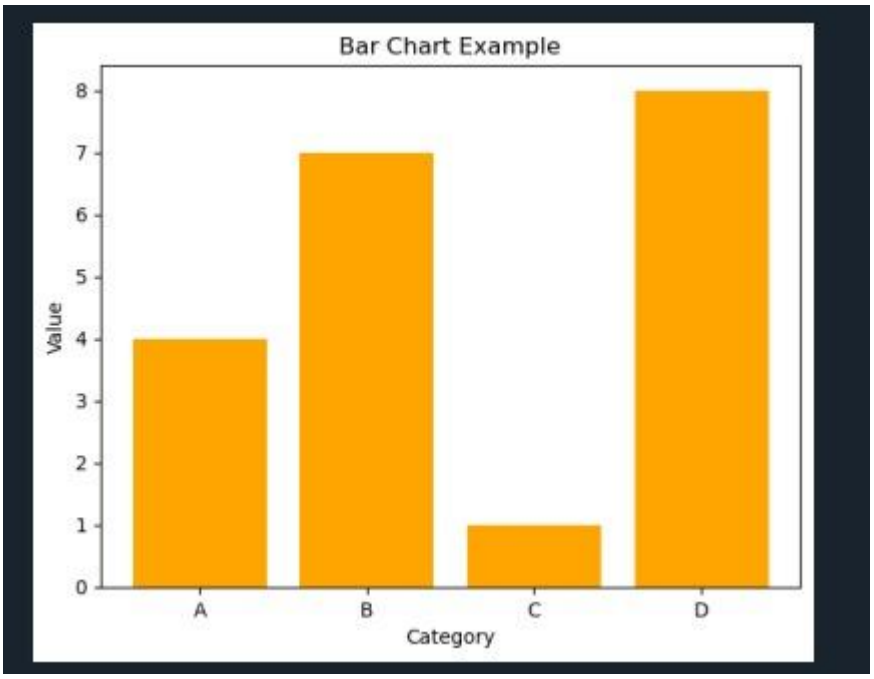
Rectangular bars are used in bar chart as the height represent the frequency of a particular element. bar() is used for plotting bar graphs, it has parameters like x, y, width, color.

bar(x, y, color, width)

Code snippet:

```
import matplotlib.pyplot as plt
categories = ['A', 'B', 'C', 'D']
values = [4, 7, 1, 8]
plt.bar(categories, values, color='orange')
plt.title("Bar Chart Example")
plt.xlabel("Category")
plt.ylabel("Value")
plt.show()
```

Output:



Description:
random.rand(5) generates a random array.

Here bar() is used along with the parameters, width which denoted the width of the rectangular bars and color for the bars.

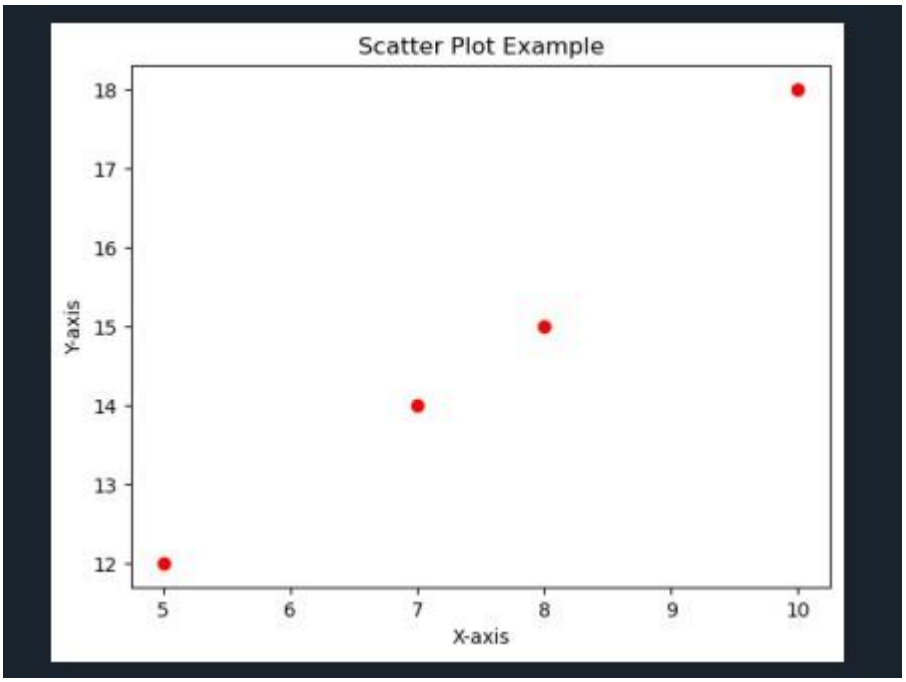
HISTOGRAM

Histogram is a type of bar graph where the graph is represented in groups. hist() is used for plotting histogram with parameters x, bins, color, edgecolor.

Code snippet:

```
import matplotlib.pyplot as plt
x = [5, 7, 8, 10]
y = [12, 14, 15, 18]
plt.scatter(x, y, color='red')
plt.title("Scatter Plot Example")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.show()
```

Output:



Description:

hist() have parameters x which is a random generated array around one hundred with standard deviation 50 of 100 values.

bins indicate the number of sections on x axis.

color indicates the color of rectangular bars.

edge color that divides the rectangular bars.

SCATTER PLOT

Scatter plot uses dots to depict the relationship between the data. scatter() is used for plotting scatter plot and scatter plot can be done for multiple datasets at the same time by differentiating it with colors. Legends help to achieve this difference.

Code snippet:

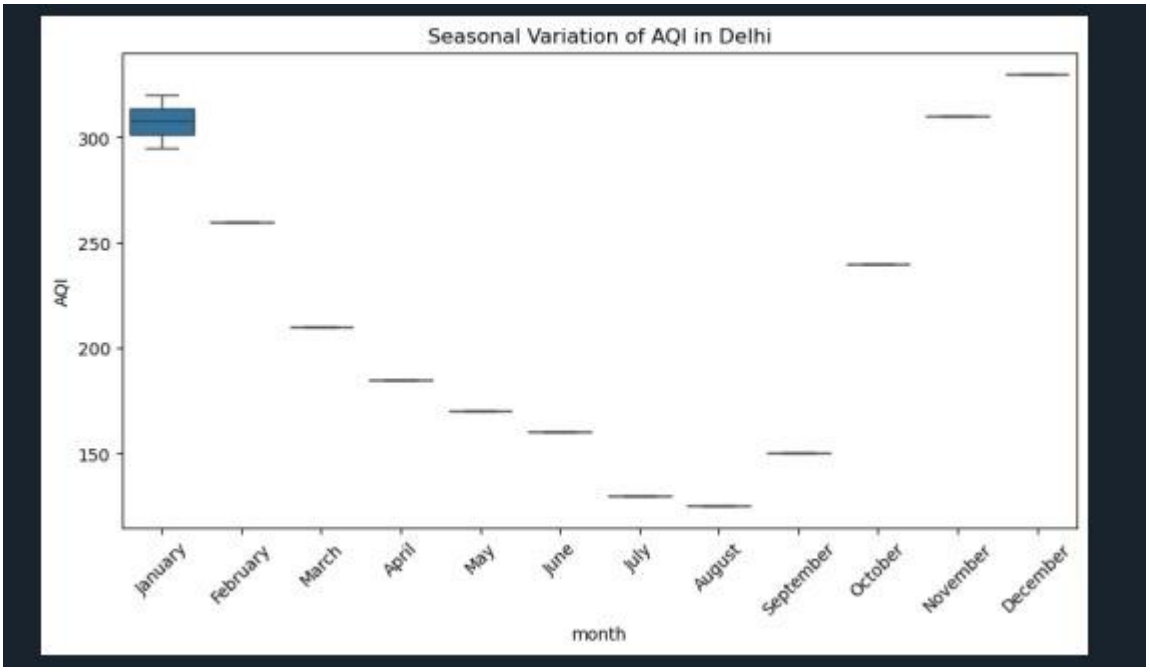
```
import pandas as pd
import matplotlib.pyplot as plt
```

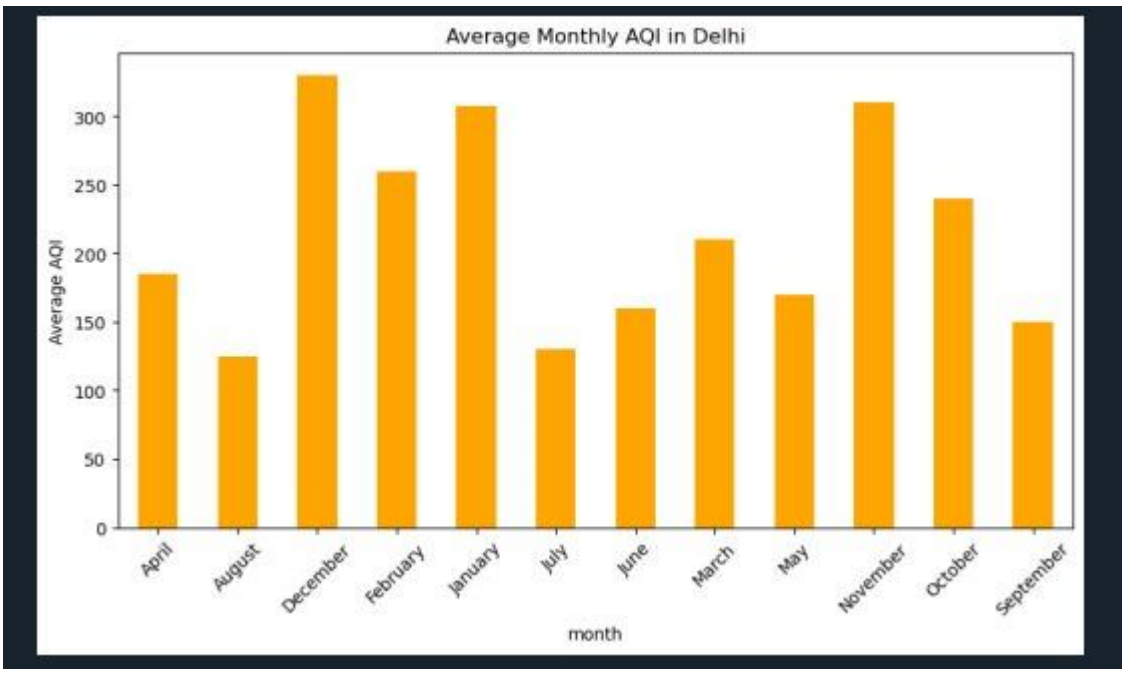
```python
import seaborn as sns

# Load the dataset (change file name if needed)
data = pd.read_csv("delhi_aqi.csv")

# Show first 5 rows
print("Dataset Preview:")
print(data.head())

# Basic information
print("\nBasic Info:")
print(data.info())

# Summary statistics
print("\nSummary Statistics:")
print(data.describe())

# Convert date column to datetime (change column name if needed)
data['date'] = pd.to_datetime(data['date'])

# Extract month and year for seasonal analysis
data['month'] = data['date'].dt.month_name()
data['year'] = data['date'].dt.year

# ---- 1 Plot overall AQI trend ----
plt.figure(figsize=(10,5))
plt.plot(data['date'], data['AQI'], color='blue', linewidth=2)
plt.title("Daily AQI Trend in Delhi")
plt.xlabel("Date")
plt.ylabel("AQI")
plt.grid(True)
plt.show()

# ---- 2 Monthly average AQI ----
monthly_aqi = data.groupby('month')['AQI'].mean()
plt.figure(figsize=(10,5))
monthly_aqi.plot(kind='bar', color='orange')
plt.title("Average Monthly AQI in Delhi")
plt.ylabel("Average AQI")
plt.xticks(rotation=45)
plt.show()

# ---- 3 Correlation between pollutants ----
pollutants = ['PM2.5', 'PM10', 'NO2', 'SO2', 'CO', 'O3']
plt.figure(figsize=(8,6))
sns.heatmap(data[pollutants + ['AQI']].corr(), annot=True, cmap='coolwarm')
plt.title("Correlation between Pollutants and AQI")
plt.show()

# ---- 4 Seasonal Variation (Boxplot) ----
plt.figure(figsize=(10,5))
```

```
sns.boxplot(x='month', y='AQI', data=data)
plt.title("Seasonal Variation of AQI in Delhi")
plt.xticks(rotation=45)
plt.show()

# ---- 5 Print simple insights ----
print("\nHighest AQI Month:", monthly_aqi.idxmax())
print("Lowest AQI Month:", monthly_aqi.idxmin())
print("Average AQI in Delhi:", round(data['AQI'].mean(), 2))
```

```
date,AQI,PM2.5,PM10,NO2,SO2,CO,O3
2024-01-05,320,180,290,42,15,1.8,24
2024-01-12,295,165,270,40,18,1.7,26
2024-02-08,260,140,230,35,14,1.6,30
2024-03-10,210,120,200,32,12,1.5,40
2024-04-14,185,110,180,28,10,1.3,55
2024-05-18,170,95,160,25,9,1.2,60
2024-06-22,160,90,150,23,8,1.1,65
2024-07-19,130,75,120,20,7,1.0,70
2024-08-10,125,70,115,18,6,0.9,72
2024-09-15,150,85,140,22,8,1.1,68
2024-10-10,240,140,230,30,12,1.4,40
2024-11-15,310,175,285,38,14,1.8,28
2024-12-12,330,190,300,45,16,1.9,25
```

OUTPUT





Description:

x1, y1 belong to one dataset and x2,y2 belong to another dataset.

Here the scatter() is used for the scatter plot and the parameters x, y and c which represents color.

Legend adds the label which helps to differentiate the plots.

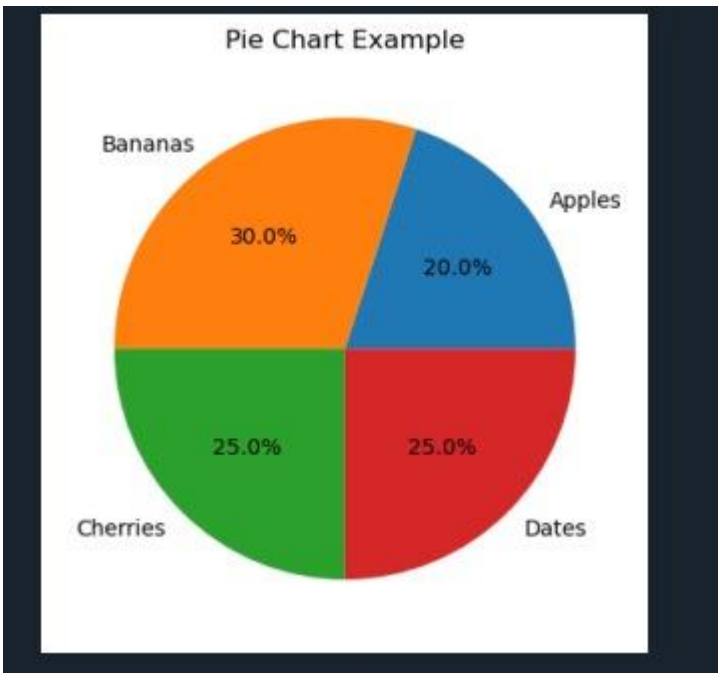
PIECHART

Pie charts are used to plot data of same kind which means the same series of data where the different elements are divide based on their percentage.

Code snippet:

```
import matplotlib.pyplot as plt
labels = ['Apples', 'Bananas', 'Cherries', 'Dates']
sizes = [20, 30, 25, 25]
plt.pie(sizes, labels=labels, autopct='%1.1f%%')
plt.title("Pie Chart Example")
plt.show()
```

Output:



Description:

Here in the code, time is the x variable and labels are defined using labels keyword.

wedgepropsattribute is used to define the linewidth and the color to separate the elements in the pie chart.

color array contains the colors to each element.

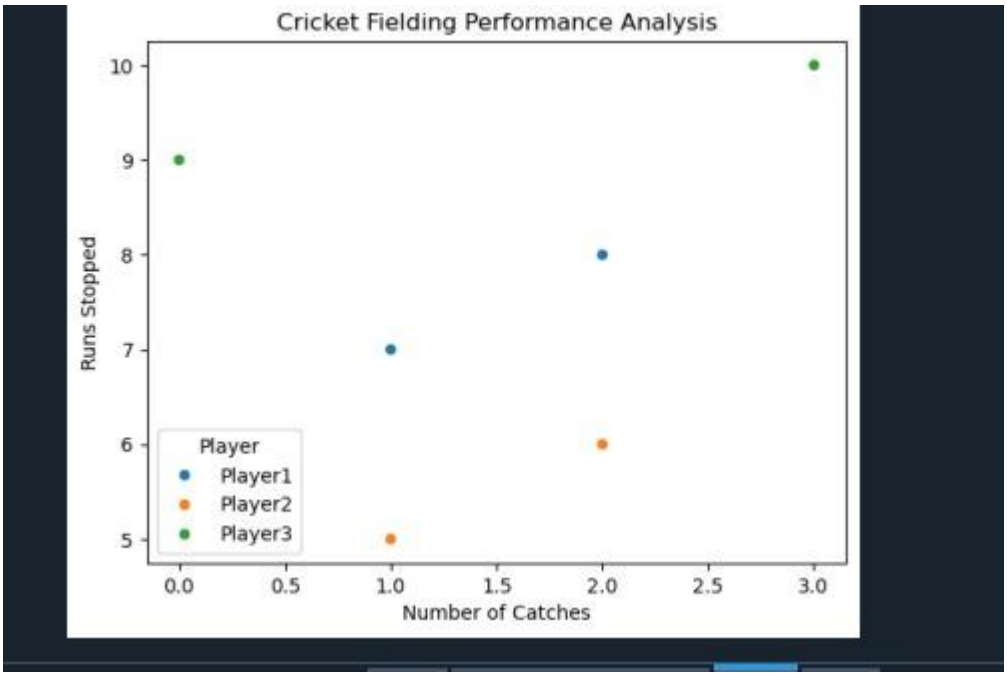
AREA CHART

In area chart, the area under the line to x axis is filled or shaded. It can be done by using fill_between() function or stackplot() function.

Code snippet:

```
import csv
filename = "cricket_fielding_data.csv"

headers = [
    "Match No.", "Innings", "Team", "Player Name", "Ballcount", "Position",
    "Short Description", "Pick", "Throw", "Runs", "Overcount", "Venue"
]
rows = [
    [1, 1, "Team A", "Player 1", "", "", "", "", "", "", "", "Stadium A"],
    [1, 1, "Team A", "Player 2", "", "", "", "", "", "", "", "Stadium A"],
    [1, 1, "Team A", "Player 3", "", "", "", "", "", "", "", "Stadium A"]
]
with open(filename, 'w', newline='') as csvfile:
```

```
    csvwriter = csv.writer(csvfile)
    csvwriter.writerow(headers)  # write headers
    csvwriter.writerows(rows)    # write empty rows

print(f"CSV template '{filename}' created successfully!")
```



Description:

fill_between() function have parameters of x and y.

stackplot() function parameters are x, y1,y1,y3 where y1, y2, y3 are the different groups.

Legends are added by mentioning the labels attribute in stackplot() function and giving the position to the legend using plt.legend() function with attribute loc.

SEABORN

Seaborn is a visualization library in python which is built on top of matplotlib. It is advanced than matplotlib and have different features which are default like style, color palette. Seaborn has different categories of plots like relational plot, categorical plot, distribution plot, regression plot, matrix plot.

Let us see the different plots in each category.

1 Relational Plots:

- scatterplot()
- lineplot()
- relplot()

2 Categorical Plots:

- barplot()
- countplot()
- boxplot()
- violinplot()
- swarmplot()
- pointplot()
- catplot()

3 Distribution Plots:

- histplot()
- kdeplot()
- rugplot()
- distplot()

4 Relational Plots:

- regplot()
- lmplot()

5 Matrix Plots:

- heatmap()
- clustermap()

Here are some sample codes for some of the graphs.

Scatter plot:

```
import pandas as pd
df = pd.read_csv("sales_data.csv")
print(df.head())
print(df[['Gross Sales', 'Net Sales', 'Profit/Loss', 'COGS', 'Manufacturing Costs',
'Freight Costs']].describe())
print(df[['Gross Sales', 'Net Sales', 'Profit/Loss', 'COGS', 'Manufacturing Costs',
'Freight Costs']].corr())
```

Output:

```
Transaction ID       Date  ...  Freight Costs  Fiscal Year
>> 0            1  2025-01-05  ...          100        2025
>> 1            2  2025-01-06  ...          100        2025
>> 2            3  2025-01-07  ...          100        2025
>> 3            4  2025-02-01  ...          100        2025
>> 4            5  2025-02-03  ...          100        2025
>>
>> [5 rows x 9 columns]
>>        Gross Sales    Net Sales  ...  Manufacturing Costs  Freight Costs
>> count     5.000000     5.000000  ...             5.000000            5.0
>> mean   1480.000000  1360.000000  ...           400.000000          100.0
>> std     396.232255   364.691651  ...            79.056942            0.0
>> min    1000.000000   900.000000  ...           300.000000          100.0
>> 25%    1200.000000  1100.000000  ...           350.000000          100.0
>> 50%    1500.000000  1400.000000  ...           400.000000          100.0
>> 75%    1700.000000  1600.000000  ...           450.000000          100.0
>> max    2000.000000  1800.000000  ...           500.000000          100.0
>>
>> [8 rows x 6 columns]
>>                   Gross Sales  Net Sales  ...  Manufacturing Costs  Freight Costs
>> Gross Sales           1.000000   0.996522  ...             0.997609            NaN
>> Net Sales             0.996522   1.000000  ...             0.997176            NaN
>> Profit/Loss           0.997609   0.997176  ...             1.000000            NaN
```

>> COGS 0.990595 0.976481 ... 0.986394 NaN

>> Manufacturing Costs 0.997609 0.997176 ... 1.000000 NaN

>> Freight Costs NaN NaN ... NaN NaN

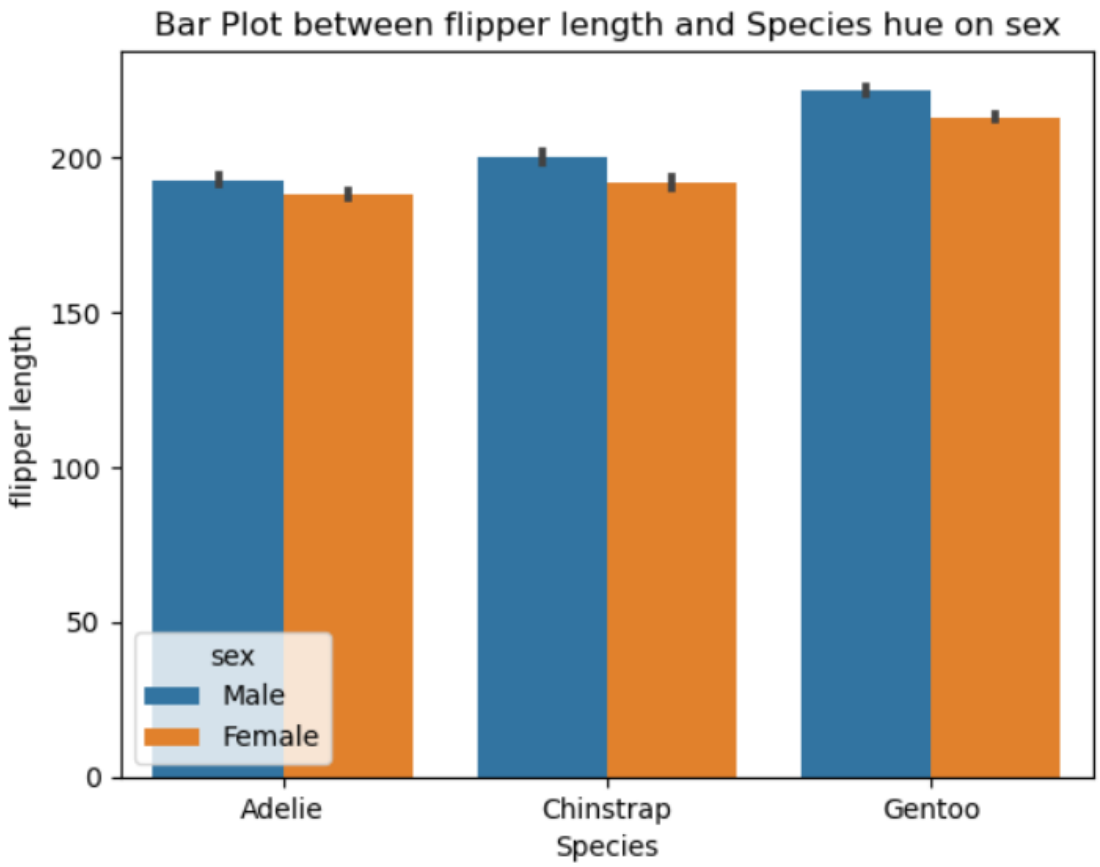
>>

>> [6 rows x 6 columns]

BAR PLOT

```
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns

sns.barplot(y='flipper_length_mm', x='species',hue='sex', data=df)
plt.title('Bar Plot between flipper length and Species hue on sex')
plt.ylabel('flipper length')
plt.xlabel('Species')
plt.show()
```

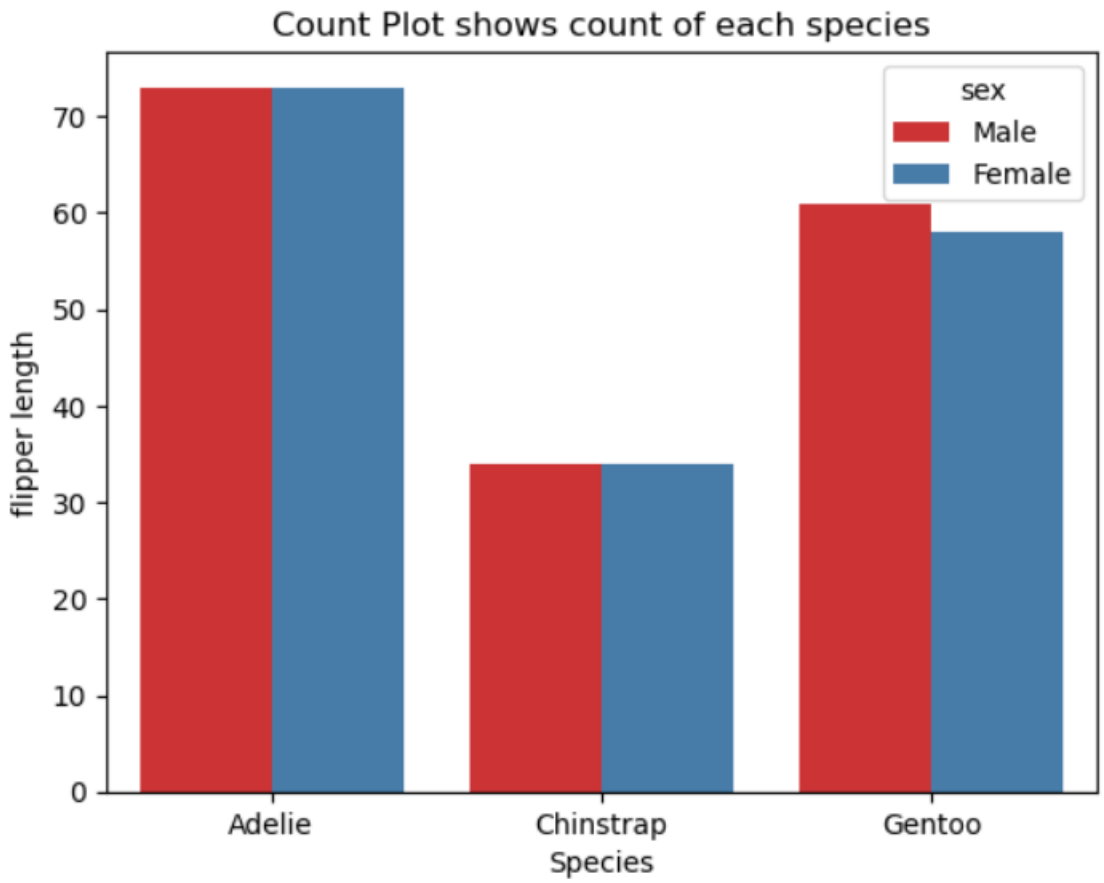
Output:



COUNT PLOT

```
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns

sns.countplot(x='species',hue='species', data=df, palette="Set1")
plt.title('Count Plot shows count of each species')
plt.ylabel('flipper length')
plt.xlabel('Species')
plt.show()
```

Output:



Description:

The count plots give the count of the data passed.

Here x is the species, hue is sex and palette as Set1.

Palette sets the color and it is Set1 is the default palette in seaborn.

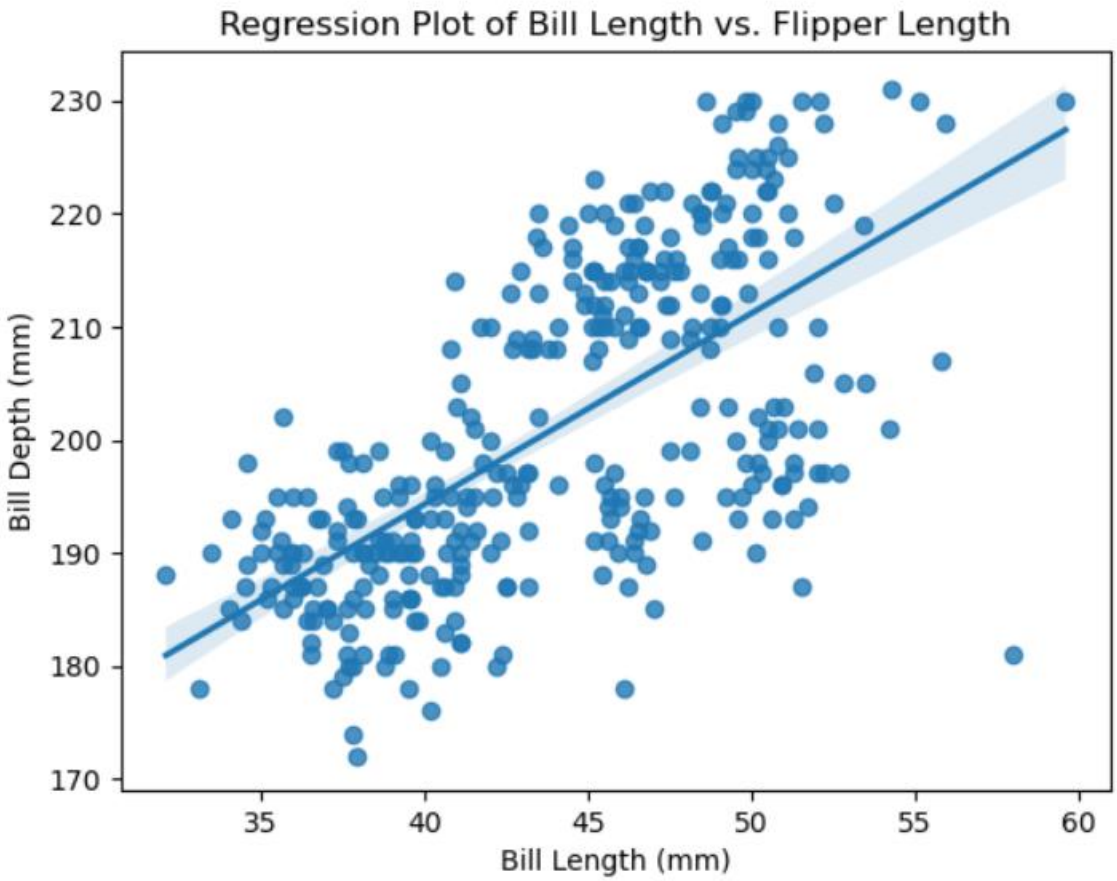
REGPLOT

Regression plots shows the relationship between variables along with the regression line.

Code snippet:

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.regplot(x="bill_length_mm", y="flipper_length_mm", data=penguins)
plt.title("Regression Plot of Bill Length vs. Flipper Length")
plt.xlabel("Bill Length (mm)")
plt.ylabel("Bill Depth (mm)")
plt.show()

```

Output:



COMPARISON OF MATPLOTLIB AND SEABORN

Matplotlib:

- Core Library: Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python. It provides a low-level interface for creating plots with fine-grained control over every aspect of the figure.
- Flexibility: Matplotlib allows users to create any type of plot imaginable, from basic line plots and scatter plots to complex 3D plots and geographical maps.
- Mature: Matplotlib has been around for a long time and is a well-established library in the Python ecosystem. Many other libraries, including Seaborn, are built on top of Matplotlib.
- Customization: Matplotlib offers extensive customization options, allowing users to customize every aspect of a plot, including colors, line styles, markers, fonts, annotations, and more.
- Integration: Matplotlib can be easily integrated with other libraries and frameworks, such as NumPy, Pandas, SciPy, which makes it a versatile choice for data visualization in various contexts.

Advantages of Matplotlib:

- High degree of customization and control over plots.
- Wide range of plot types and styles.
- Strong community support and extensive documentation.
- Well-suited for creating publication-quality figures and graphics.

Seaborn:

- High-Level Interface: Seaborn is built on top of Matplotlib and provides a high-level interface for creating attractive and informative statistical graphics. It simplifies the process of creating complex visualizations by providing default aesthetics and convenient functions for common tasks.
- Statistical Plotting: Seaborn specializes in statistical plotting and offers a variety of plot types specifically designed for visualizing relationships in data, such as scatter plots, bar plots, box plots, violin plots, pair plots, and more.
- Aesthetics: Seaborn comes with built-in themes and color palettes that make it easy to create visually appealing plots with minimal effort. It also provides tools for fine-tuning plot aesthetics, such as controlling the size of plot elements and adjusting the color palette.
- Integration with Pandas: Seaborn seamlessly integrates with Pandas dataframes, allowing users to easily plot data stored in Pandas data structures without the need for extensive data manipulation.

Advantages of Seaborn:

- Simplified syntax and high-level functions for creating complex plots.
- Built-in support for statistical plotting and data exploration.
- Attractive default aesthetics and color palettes.
- Seamless integration with Pandas dataframes for data visualization.
- Ideal for exploratory data analysis and quick visualization of relationships in data.

Overall, the choice between Matplotlib and Seaborn depends on the specific requirements ofdata visualization task. Matplotlib is preferred for its flexibility and fine-grained control over plots, while Seaborn is preferred for its ease of use, statistical plotting capabilities, and attractive default aesthetics. Many users combine both libraries, leveraging the strengths of each as needed.